ZRA Labs 2026 - Railway Asset Classification

Railway Asset Classification for Edge Deployment

ZRA Labs Project Whitepaper 2026

Author	Abhiram Vadali
Mentor	Venkata Phani
Project Coordinators	Chaitanya Mokrala, Railway Academy Sumit Kanu, Railway Academy
Programme	ZRA Labs Summer Internship 2026
Date	July - August 2026

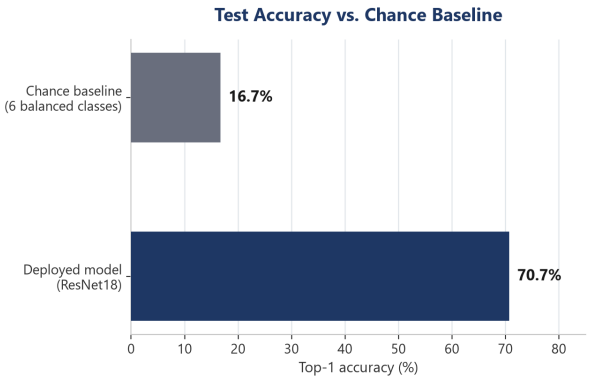
Table of Contents

I. Executive Summary

A convolutional classifier built during this internship sorts photographs of railway infrastructure into one of six categories: train, track, signal, platform, crossing gate, and overhead wire. It reached 70.7% top-1 accuracy in testing, against a chance baseline of 16.7%. The initial prototype ran as a Streamlit app, which meant it needed either constant network access for remote processing or a capable discrete GPU to run offline, and neither is readily available during inspection in remote regions. This report looks at what it takes to run the same pipeline on hardware carried to the track, where the device has to run inference against a locally stored model, keep the resulting records, and hand them back to the user, all on the edge device itself.

Two constraints came out of the analysis. The microcontroller boards proposed for drone mounting, Arduino and ESP-32, cannot hold a model of this type. The smallest architecture that can do the classification overruns their memory by roughly an order of magnitude, so those boards are limited to capturing images and transmitting them, or to a narrow binary detector trained separately for the job. ResNet18, the architecture currently deployed, also costs far more than the task needs. When testing on a mid-tier laptop (Ryzen 5 5500U), a full sliding-window scan of a 1280×960 frame took 2204 ms with ResNet18 and 615 ms with MobileNetV3-Small, which has about a fifth of the parameters.

MobileNetV3-Small is therefore the lead candidate for phones, tablets, and low-power computers, running under ONNX Runtime where cross-platform portability matters, TFLite on boards like the Raspberry Pi 4B, and ExecuTorch where the target is a handset. Section VI sets out the accuracy comparison, and Section VII covers the physical validation.



Test accuracy vs. chance baseline (original 351-image dataset)

II. Introduction

This project began under ZRA Labs' AI in Rail program area. The project was originally a computer vision demonstration: a system that could accept an uploaded railway photograph and tag it as containing a train, track, signal, platform, overhead wire, or crossing gate. The early stage of this project was not a full industrial-grade inspection system out of the gate, only a working prototype showing how computer vision could support education & safety in rail. What was built went past that starting point. After the initial task was completed, a question of whether that same classifier could run without a hosted backend (on edge-devices) at all. The remainder of this report addresses that question, running the model on hardware carried into the field instead of a server.

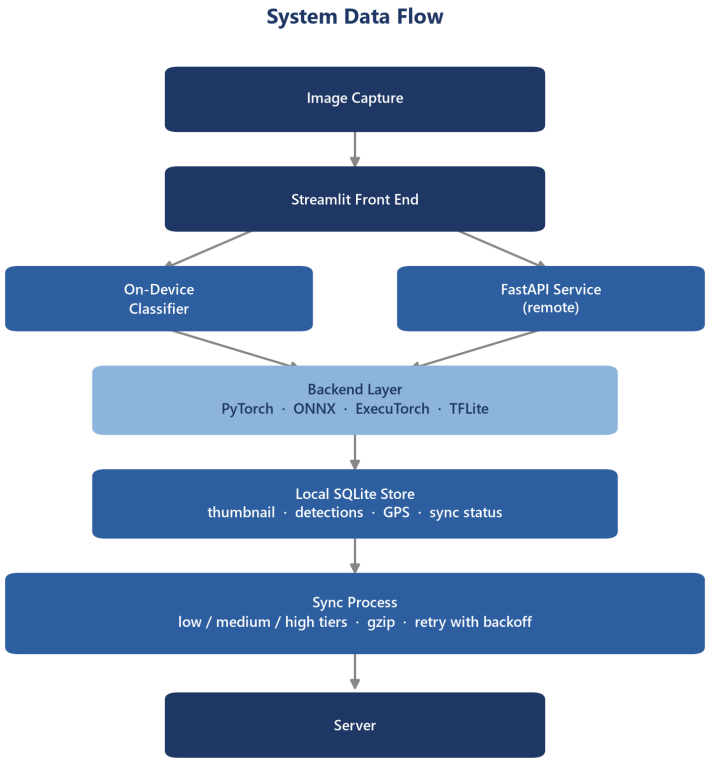
Scope and Objectives

Initially, the project called for using a pre-trained model, and moving on to a custom-trained one, built from thirty to fifty images per category, if time permitted. This work had moved past both tiers, training on a larger set than the advanced version called for, and reaching a working accuracy figure rather than a rough proof of concept. It covers what the current system does and where that design depends on a network or a discrete GPU, what would have to be true for it to run disconnected on smaller hardware, which smaller architectures could substitute for the deployed model and how they map onto specific device tiers, and how captured data would move once a device is offline by design rather than by accident. It does not cover procurement, physical mounting to a drone or vehicle, or a redesign of the demonstration interface itself; those sit outside what this report sets out to answer.

Intended Audience & Uses

This report is meant to inform decisions inside the AI in Rail program about whether and how to carry this demonstration project toward an on-device version, and to leave a clear technical record for whoever continues this specific project line in a future cycle. The original brief noted that a working demo could feed later into AI in Rail course material and school outreach activities. It assumes basic familiarity with machine learning terms but not with the codebase itself, since the goal is to communicate what exists and what is missing, not to document how any particular part is written.

III. System Overview

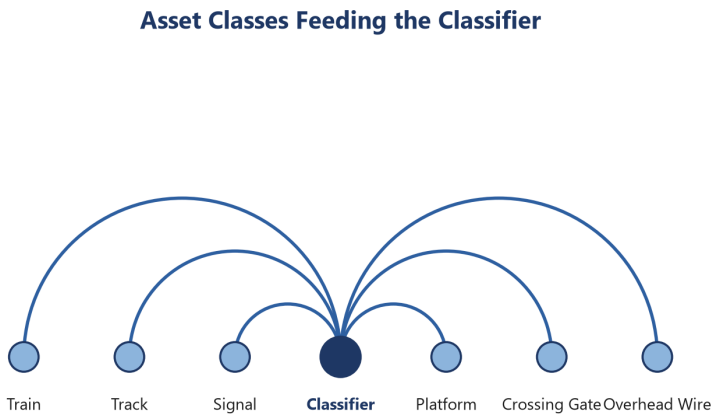


Architecture Diagram

RailwayClassifier wraps a swappable TorchVision backbone. It defaults to ResNet18 [3], with five smaller alternatives recorded in the checkpoint, so changing architectures is a config value rather than a code change. Since the network only classifies whole images, finding multiple assets in one frame means scanning it as overlapping crops at a density set by a named ScanProfile, then merging the overlapping hits into single boxes. Inference runs through a backend layer that supports PyTorch, ONNX, ExecuTorch, and TFLite. The Streamlit front end can run locally or call a FastAPI service, while a local SQLite store logs every capture and syncs it to the server in zipped batches when a connection is available [1]. Each capture is saved as a downscaled thumbnail with its detections and, when available, GPS coordinates pulled from the photo's EXIF [2]. sync.py pushes these to the server in tiered batches, from metadata only on a narrowband link up to full thumbnails on a fast one, retrying with backoff instead of dropping data.

Asset Classes in Scope

The model was trained on six classes: train, track, signal, platform, crossing gate, and overhead wire. The training set came from a script that searched Wikimedia Commons using just a handful of tags per class, which yielded 351 original photographs (roughly 60 per class, after Week 4's growth pass), expanded to a 980-image training set through data augments. These included cropping, flips, colour jitter, blur, and rotation. Overhead wire needed its own script, one that cropped down to sky and wire alone. An earlier attempt trained on uncropped images had collapsed into a wire-versus-everything classifier. Against a 16.7% chance baseline across six balanced classes, the current checkpoint reaches approximately 70% top-1 accuracy in benchmarking.



The asset classes that the model was trained on.

Limitations of a Server-Hosted Approach

This system runs three ways today, and two of them assume a network: a hosted classifier that turns every scan into an HTTP round trip, and the public demo on Streamlit Community Cloud, which carries the same dependency in a different form. The third is on-device inference, which the interface falls back to automatically when a request fails, since a network cannot be assumed at track-side. That fallback removes the network dependency but not the hardware one: it still needs a full machine learning runtime installed, and it benefits heavily from an accelerator, checking for a GPU or other available hardware before falling back to the CPU. ResNet18, at 11.7M parameters, is the heaviest of the six backbones on offer, and it's the one currently deployed. Training time dropped from about 19 minutes to 5 after moving from a laptop CPU to a discrete GPU (RTX 3060 Ti). Neither a reachable server nor a discrete GPU can be assumed at track-side or on a drone in a remote region.

IV. Edge Deployment Requirements

On-Device Classification

Running inference on the device means the classifier has to load, run, and return results without a network connection. The backend layer already supports this: it can run through PyTorch, ONNX, ExecuTorch, or TFLite. ONNX Runtime is the choice where cross-platform portability matters most, laptops and desktops included; TFLite is the better fit for low-power boards such as the Raspberry Pi 4B; ExecuTorch [5] is built specifically for handsets. The classifier can switch between six backbones, from ResNet18 down to SqueezeNet1.1, and the checkpoint records which one is loaded. Because the network classifies a single image at a time, detecting multiple objects in a frame means scanning it as a grid of overlapping crops. Three scan profiles control how dense that grid is, including a full sweep and an edge profile that does roughly an eighth of the work. Accuracy has now been measured for all six backbones (Section VI): the smaller ones trail ResNet18's 75.8% on the recollected test set, with MobileNetV3-Small reaching 61.6%, a gap the device-tier recommendations below have to weigh against speed and footprint rather than close outright.

Local Storage of Images and Metadata

Every captured image is written to a local database immediately, so the picture and its detections survive even on days the device never reaches a server. Only a downscaled copy is kept, with the long side capped at 640 pixels. Alongside that thumbnail, the database stores the labels, confidence scores, and bounding boxes for each detection, plus a timestamp and, when the photo's EXIF data includes one, a GPS location. Each device generates its own unique ID once, so records from a fleet of similar devices can be told apart later. Every entry also carries a sync status: it stays pending until the server confirms receipt, which is how the device tracks what it still has to send. The database uses a write-ahead log [12], so the sync process can read the outbox while a new photo is still being written. None of this depends on the classifier or which runtime it uses, deliberately: the storage layer runs on whatever hardware holds the camera, and that's the hardware least likely to have a machine learning runtime installed.

Cloud Transmission for Analytics and Telemetry

Getting images off the device is handled by a sync process, sending unsynced captures in batches. On a narrowband radio link that might mean ten records at a time; a faster connection can push up to fifty, thumbnails included, if there's room. The smallest tier sends only metadata, no images at all. Each batch is compressed before it leaves the device, and a batch that fails to go through is retried after a delay rather than dropped, with the delay staggered so that multiple devices coming online at once don't all retry in the same instant. A record only counts as sent once the server confirms it; until then, the device keeps it. That way a lost response doesn't cost a capture, and the same capture never gets sent twice.

V. Constraints for On-Device Inference

Compute and Memory Budgets

Every backbone in this pipeline was chosen from a narrow band of options, ranging from ResNet18's 11.7 million parameters down to SqueezeNet1.1's 1.2 million. Even the smallest overruns the memory of the microcontroller boards proposed for drone mounting by roughly an order of magnitude, which rules Arduino and ESP-32 class boards out of running the classifier at all, leaving them limited to capturing and transmitting images or running a narrow binary detector trained separately for the job. How large a scan batch can be is tuned per device: the full scan profile processes 32 crops together on a workstation, while the edge profile shrinks both the batch and the scan grid for boards with far less headroom. Every megabyte spent on the model is a megabyte unavailable for the image buffer or the crop grid, which is why the smallest workable backbone, not the most accurate one, is the right default for constrained hardware.

Power and Thermal Envelopes

Power is scarcer than compute on most of this hardware, since anything mounted to a drone draws from the same battery that keeps it airborne, and every milliamp spent on inference is one not spent on flight time. A microcontroller idles at a few hundred milliwatts so it can run for a full inspection pass, but that budget leaves no room for a convolutional network of any size. A single-board computer or phone can absorb the classifier's load, though a sustained sliding-window scan, dozens of forward passes back to back, draws more current and generates more heat than a single classification would. Most of these boards are passively cooled, so a long scanning session risks thermal throttling, which quietly lowers clock speed and slows every subsequent inference rather than failing outright. A device asked for one classification tolerates this fine; one asked to sweep a full frame repeatedly over a long route needs a scan profile and schedule that keep it under its enclosure's thermal ceiling.

Implications for Model Size

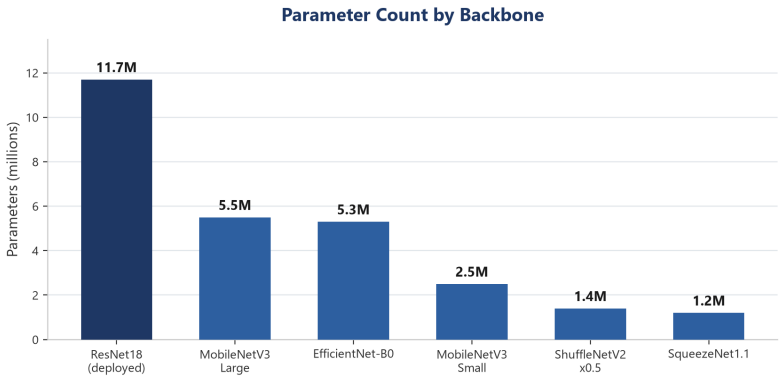
Model size was the primary factor in deciding on the recommended edge-hardware for this project. A parameter count in the tens of millions suits a laptop or a Jetson, but it forecloses the microcontroller tier and pushes a phone or single-board computer towards thermal limits during a long scan. The smaller backbones (MobileNetV3-Small, ShuffleNetV2, and SqueezeNet1.1 among them) exist specifically for this tier, trading capacity for a footprint that fits a constrained device's memory and power budget. A smaller checkpoint also matters for updates, since a device pulling a model over a slow link benefits from that file being tens of megabytes rather than over a hundred. That trade is now measured (Section VI), and it is not a clean function of size. MobileNetV3-Large holds 67.7% at 5.5 million parameters and MobileNetV3-Small holds 61.6% at 2.5 million, so most of the cut costs about six points. Below that the results stop tracking parameter count at all: SqueezeNet1.1 reaches 55.6% on 1.2 million parameters while

the larger ShuffleNetV2 x0.5 manages only 42.4% on 1.4 million. Architecture, not footprint, decides what a backbone is worth at this scale, which means each candidate has to be measured on this task rather than ranked by size.

VI. Candidate Models for Edge Inference

Small-Footprint Model Families

Beyond the deployed ResNet18, five smaller backbones, all pretrained on ImageNet [7], are already recorded in the classifier and ready to load in its place. MobileNetV3 [8] comes in two sizes, 5.5 million parameters for Large and 2.5 million for Small, built from depthwise separable convolutions and squeeze-and-excite blocks. EfficientNet-B0 [4], at 5.3 million parameters, scales depth, width, and resolution together. On this six-class task it scored 60.6%, below MobileNetV3-Small, which has less than half its parameters. ShuffleNetV2 [9] drops to 1.4 million by shuffling channels between grouped convolutions, and SqueezeNet1.1 [11] goes smaller still, at 1.2 million, by squeezing down before expanding back out. All five swap in the same way the current backbone loads, so retraining on one is a configuration choice, not a rewrite. Measured accuracy for all six follows in the next section.

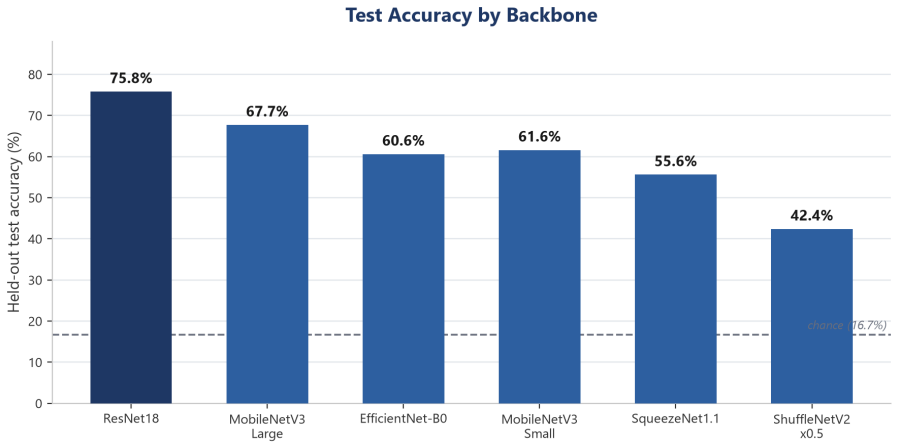


Parameter Count for Each of the Six Backbones

Optimisation Techniques

Swapping to a smaller backbone is the optimisation this pipeline has actually used so far, as quantisation and pruning have not been implemented yet. Some quantisation support exists further down the pipeline: the interpreter for the smallest boards can read an integer model directly, rescaling its output using the scale and offset the export step recorded. That covers running an already-quantised model, not producing one. Pruning or distillation could compound with a smaller backbone rather than substitute for one. Until quantised or pruned, the size figures here describe full-precision weights only.

Accuracy vs. Footprint Trade-offs



Accuracy across all six backbones on the 633-image recollected dataset.

VII. Matching Models to Devices

Low/Mid-Capability Device

A low/mid tier device would be a Raspberry Pi 4B in this case, running the classifier on the onboard CPU. The Pi 4B should be capable of running the classifier entirely on-edge, at track-side in remote regions. It is also slow enough on a sustained scan to risk thermal throttling if the workload isn't kept modest. The right combination follows directly: one of the three smallest backbones, MobileNetV3-Small, ShuffleNetV2, or SqueezeNet1.1, chosen less for memory, since even the largest of the six fits the Pi's RAM easily, and more because a smaller network means fewer milliseconds per crop on a CPU with no help. TFLite is the natural runtime here, skipping the full framework a Torch install would need. Pairing it with the edge scan profile keeps the sweep to the smallest workable number of passes, trading recall for a scan the board's cooling can sustain. It needs no accelerator and no quantised export, just the existing checkpoint on hardware this report has already covered, making it the pairing closest to testable today. Of the three, MobileNetV3-Small has the best measured accuracy at 61.6%, though that figure comes from a laptop benchmark (Ryzen 5 5500U) and still needs validating on the Pi itself.

Accelerator-Equipped Device

In theory, the same Raspberry Pi 4B could be fitted with a Coral Edge TPU accelerator, cutting scan time and allowing multiple images to be processed in tandem. The blocker is not the model format, since the backend layer already exports TFLite. It is quantisation. The Edge TPU runs int8 models only, and the pipeline can load a quantised model but cannot produce one, so this

pairing stays theoretical until a quantised export path is built. The accelerator itself draws only a few watts over USB, modest enough that it would not reopen the power budget problem a discrete GPU would. What it would buy is speed: a quantised model on a dedicated accelerator runs a scan in a fraction of the time the bare CPU needs, likely enough to afford the full or balanced profile instead of the edge one.

VIII. Data Handling and Transmission

Local Storage Format & Retention

Every capture lands in a local database as soon as it is scanned, using a write-ahead log so a write in progress does not block whatever is reading the outbox for sync. Only a downscaled copy survives, its longest edge capped at 640 pixels and saved as a JPEG, keeping full-resolution photographs off the device's storage. A record's sync status changes from pending to synced once the server has acknowledged it, but nothing currently removes a synced capture, so a database running for months holds the same number of thumbnails whether they were sent yesterday or a year ago. On a low-cost edge device such as a phone or single-board computer, unbounded growth is a real constraint the current design leaves open. A retention window, keyed off sync status and age, is the natural next step once this becomes the bottleneck it is not yet.

Metadata Schema

Each capture record carries an identifier, the device it came from, a timestamp, and the filename of the source image, alongside which model and version produced the result. Where the photo's own file data included a location, that is pulled out and stored alongside the rest rather than left buried in the image. The detections themselves live as their own set of rows tied back to the capture that produced them, one row per object found, each carrying its label, a confidence score, and the box it was found in. A handful of fields exist purely for the device's own bookkeeping and are stripped out before anything is sent onward: the sync status, when it was last confirmed synced, and the path and size of the saved thumbnail. That split, what travels and what stays local, is what lets the same record serve two purposes without carrying weight neither one needs.

Low- and Medium-Bandwidth Sync Strategies

Two of the three sync tiers are built for links too slow to carry images. The lower one sends ten records at a time and metadata only, no thumbnail, on the assumption that a narrowband radio can carry a label, a confidence score, and a timestamp far more reliably than a picture. The medium tier raises the batch to twenty five and adds the thumbnail back in, sized for a connection fast enough to spare the extra bytes without falling behind the capture rate. Both compress the batch before it leaves the device, since the fields being sent, mostly short strings

and numbers, shrink considerably under compression even before a thumbnail is added. Neither tier assumes the batch arrives on the first try: every batch is retried with a growing delay if it fails, so a connection that comes and goes still eventually gets everything through rather than losing whatever showed up while it was down.

Failure Modes and Offline Behaviour

The system is built to degrade rather than stop working when something upstream is unavailable. A request to classify a photo that cannot reach the server falls back to running the model on the device itself, with a warning shown rather than an error, since the whole reason this runs offline is that the network is the part expected to fail. A capture that cannot be written locally, because storage is full or read-only, is caught the same way: the scan result still reaches the screen even though it will not be recorded. On the sending side, a batch the server rejects outright, a malformed request, is not retried, since sending it again would not fix it, but a batch that times out or gets a rate-limit response is retried with a growing, randomised delay [6], the same pattern behind most production retry logic [10], so several devices coming back online together do not all retry in the same instant. A batch the server has already seen is acknowledged again rather than rejected, so a reply lost on the way back does not leave a capture stuck being resent forever.

IX. Conclusion

ZRA required a demo tagging images as train, track, signal, platform, overhead wire, or crossing gate, with a confidence score and one custom model compared against one pre-trained model. This project measured test accuracy for six backbone architectures and supports four runtime formats. CPU latency was measured for two of the six. ResNet18 leads at 75.8%; ShuffleNetV2 x0.5 is the weakest of the six at 42.4%, still well above the 16.7% chance baseline but too far below the rest to recommend. On a Ryzen 5 5500U, a full scan took 2,204 ms with ResNet18 and 615 ms with MobileNetV3-Small, a sublinear speedup. The pipeline can load a quantised model but not produce one, blocking the Raspberry Pi 4B plus Coral Edge TPU pairing, and pruning and distillation of the model are untried. ZRA should test MobileNetV3-Small on a bare Raspberry Pi 4B under TFLite before committing to any accelerator hardware, since that pairing runs on the existing checkpoint and needs nothing that this project has not already built. The quantised export path is the next piece of work after that: it is what the Coral Edge TPU pairing waits on, and nothing about the accelerator tier can be measured until it exists.

X. References

[1]

K. Fall, “A Delay-Tolerant Network Architecture for Challenged Internets,” in *Proceedings of the 2003 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM)*, 2003, pp. 27–34. doi: 10.1145/863955.863960.

[2]

Camera & Imaging Products Association, “CIPA DC-008-2019: Exchangeable Image File Format for Digital Still Cameras: Exif Version 2.32,” CIPA, 2019.

[3]

K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.

[4]

M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

[5]

PyTorch Team, “ExecuTorch.” 2024. [Online]. Available: https://pytorch.org/executorch/

[6]

M. Brooker, “Exponential Backoff and Jitter.” 2015. [Online]. Available: https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/

[7]

J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255. doi: 10.1109/CVPR.2009.5206848.

[8]

A. Howard *et al.*, “Searching for MobileNetV3,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.

[9]

N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, “ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.

[10]

B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O’Reilly Media, 2016.

[11]

F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size.” 2016.

[12]

SQLite Documentation, “Write-Ahead Logging.” 2024. [Online]. Available: https://www.sqlite.org/wal.html